Forward Deployed Engineering from First Principles to Enterprise Delivery

Course syllabus · version 3 · approved 2026-10-26

Who it's for: Software engineers who can write functions and call a web API; no ML background assumed. Some Python is needed.

What you'll be able to do: Scope a customer problem, integrate with its systems, build and evaluate an AI feature, deploy it into a restricted customer environment, hand it over, and explain the trade-offs in an interview.

33 lessons · about 15 hours

Interview questions & projects: Interview questions (interview/INTERVIEW_QUESTIONS.pdf) · Interview answer key (instructors only) (interview/ANSWER_KEY.pdf) · Basic projects (projects/BASIC_PROJECTS.pdf) · Intermediate projects (projects/INTERMEDIATE_PROJECTS.pdf) · Advanced projects (projects/ADVANCED_PROJECTS.pdf)

Tools you'll use

Tool	What for	Lessons	How you can run it
Gemini on Vertex AI (llm_setup)	the model behind the LLM lessons	06, 11, 12, 18, 19, 20, 28, 31, 32	Gemini API key (default)
Docker Compose	the customer environment: network, proxy, services	14, 15, 16, 20, 24, 28, 31, 32	local (Docker, no account)
FastAPI	the mock customer API and the course app	04, 13, 16, 20	local (no account)
Pydantic	checked structured output and settings	06, 19	local (no account)
pytest	regression tests for evals and APIs	04, 11, 12, 32	local (no account)
OpenTelemetry (Python)	traces across the model call	16, 20	local (Collector, Jaeger, Prometheus in Compose)

You'll be asked which option you want, and for any keys, just before the lessons that need it.

Before you start

• **GOOGLE_CLOUD_PROJECT** — Alternative to the API key: your Google Cloud project id and location (lessons 06, 11, 12, 18, 19, 20, 28, 31, 32).
• **GOOGLE_APPLICATION_CREDENTIALS** — Path to a service-account JSON file, if you use one instead of an API key (leave empty otherwise).
• **TUT_LLM_MODEL** — Model id the LLM lessons use (lessons 06 onward). Keep it on the cheapest current model.
• **TRIAGE_SERVICE_TOKEN** — Lesson 13 (app step 3): the token callers send to the triage app's /api routes as 'Authorization: Bearer <token>'. Make one: python -c "import secrets; print(secrets.token_urlsafe(32))". Empty means the app answers 503 on /api.
• **OTEL_EXPORTER_OTLP_ENDPOINT** — Lesson 16: the OpenTelemetry Collector the triage service sends its traces to (inside the Compose network; a customer sets its own collector here).
• Docker installed (some lessons start services in containers)
Put the values in this course's .env file (template: .env.example).

Contents

Basic — 9 lessons · ~4 h

Chapter 1. Basic: the customer problem and the first working proof of concept

01 Why forward deployed engineering, and when you don't need it · theory — Explain what a forward deployed engineer does, how the role differs from software engineer, solutions architect and consultant, and when a product alone is enough.

02 Finding the real problem: discovery and decomposition — Run a discovery interview, split a vague request into problems you can test, and read a customer's systems before writing code.

03 Scoping a proof of concept and a two-week sprint · walkthrough — Write success criteria and a scope that fits a sprint, and say what the proof of concept will not do.

04 Integrating with a customer's API: auth, pagination, retries, idempotency — Call a customer's REST API safely with a token, page through results, retry on failure without duplicating writes, and debug a broken call.

05 Ingesting customer data: files, pipelines and deduplication — Load a messy customer export, remove duplicates by a stable key, and write the result in a form the customer's system can read.

06 A customer workflow with structured output and tool calling — Use a model to turn a customer's free-text request into a checked record with tools, and decide when a model is the wrong tool.

07 Explaining progress to non-engineers and running a live demo · walkthrough — Give a status update and a live demo to a customer's operations lead, and handle a question you can't answer yet.

08 Timelines, risk and success metrics for a customer engagement — Plan an engagement's timeline, list its risks with owners, and agree measurable success metrics with the customer.

09 Checkpoint: a working proof of concept for one customer workflow ★ — Build the scoped proof of concept end to end against the mock customer API, and show it to a non-engineer.

Intermediate — 11 lessons · ~5 h

Chapter 2. Intermediate: evaluate, secure, deploy and observe

10 Why customer AI needs evals: failures you can't see · theory — Explain why a customer AI feature fails quietly, and what an eval catches that a demo never shows.

11 Build an eval set from real failures — Turn real failure cases from a customer into a small eval set and a regression test in pytest.

12 Grading outputs with code, model and human graders — Grade model outputs with a code check, a model grader and a human review, and compare their agreement.

13 Authentication, secrets and least privilege in a customer system — Use a service account with a scoped token, keep secrets out of code, and check who can call what.

14 Deploying into a customer environment with Docker Compose — Package the service in a container, deploy it with Compose, and roll back a bad release.

15 Deploying into a restricted network: private networks and an egress proxy — Run the service on a Compose network with no direct internet access and reach the model only through an egress proxy.

16 Observing a customer system: logs, traces and metrics — Add structured logs, a trace across the model call and a metric for latency, and find a slow request from them.

17 Runbooks and incident response for a customer deployment · walkthrough — Write a runbook for the three most likely incidents, including a restore drill from backup.

18 Cost, latency and caching: budgets for a customer's AI features — Set a token and latency budget, measure both, and cache repeated lookups with an LRU cache.

19 Agent workflows for a customer: tools, MCP servers and guardrails — Give an agent a small set of tools through an MCP server, bound its steps and check its guardrails.

20 Checkpoint: an evaluated, deployable proof of concept ★ — Deploy the proof of concept with an eval gate, auth and observability, and show the eval results to a customer.

Advanced — 13 lessons · ~6 h

Chapter 3. Advanced: production, handover, productizing and delivery

21 From proof of concept to production: what changes · theory — Name what a proof of concept lacks for production, and compare published case studies using only numbers from their sources.

22 Handover: documentation and enabling the customer's team — Hand a deployment to a customer team with docs and a training session they can run without you.

23 Per-customer configuration and multi-tenancy — Run one service for several stores with isolated order files and per-store limits, with CSV and Excel as the only storage.

24 Zero-downtime changes: rolling upgrades and migrations — Roll out a new version and a new column in the orders file to a running deployment with no downtime and a rollback path.

24a Long batch jobs: restart, reconcile and finish on time — Run a nightly batch of about 1.2 million orders against a deadline on two nodes, restart it after a crash without duplicate or missing rows, and check its pace in order when it falls behind.

25 Productizing field work: reusable components, custom vs core — Decide what to keep as one customer's code and what to turn into a shared component, then refactor the broken code safely.

26 Feedback loops to product: writing a field report · walkthrough — Write a field report that turns a customer's repeated request into a product issue with evidence.

27 Security reviews and compliance evidence · walkthrough — Answer a customer's security questionnaire with evidence: data residency, PII handling and audit logs.

28 Failure drill: a model provider goes down — Break the model provider on purpose under load, watch errors and latency, and recover with a fallback and a capped retry per call (a shared budget is a platform decision).

29 Cost engineering: unit economics for a customer deployment — Work out cost per customer and per query, and find the two biggest cost levers.

30 Ambiguity and the business case: saying no and saying what it is worth · theory — Write a business case for one customer proposal, say no to an out-of-scope request, and reset expectations after a missed date.

31 Delivering one customer deployment end to end — Take one scoped customer problem from discovery to a deployed, documented, evaluated service.

32 Final checkpoint: the customer delivery drill ★ — Run the full delivery drill against a fresh customer scenario, in a time box you set yourself, and present it.

★ = project lesson

Industry reading

How companies that run this at scale do it: one to three posts per chapter.
Ch 1 · Building effective agents — Anthropic Engineering, 2024-12. Start with the simplest workflow that works and add agent complexity only when it measurably helps: the same discipline that keeps a two-week proof of concept scoped and demoable.
https://www.anthropic.com/engineering/building-effective-agents
Ch 1 · Designing robust and predictable APIs with idempotency — Stripe, 2017-02. Classic (2017) and still the reference for idempotency keys and retry with backoff and jitter, the exact habits needed when integrating with a customer's API; no newer first-party write-up replaces it.
https://stripe.com/blog/idempotency
Ch 1 · Timeouts, retries, and backoff with jitter — Amazon Builders' Library (AWS), 2019-12. Classic (2019) by Marc Brooker: why unbounded retries turn a slow customer API into an outage, and how capped backoff with jitter prevents it.
https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/
Ch 2 · Demystifying evals for AI agents — Anthropic Engineering, 2026-01. How to build, grade and maintain automated tests for agents so regressions show up before the customer sees them; maps to building the eval set and choosing code, model and human graders.
https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
Ch 2 · Creating custom LLM judges to improve AI agent accuracy — Databricks, 2025-11. A vendor that deploys AI for enterprises shows judges tuned with expert feedback and checked against human raters: the practical way to trust a model grader.
https://www.databricks.com/en/blog/building-custom-llm-judges-ai-agent-accuracy
Ch 2 · Beyond permission prompts: making Claude Code more secure and autonomous — Anthropic Engineering, 2025-10. Filesystem and network isolation with an egress proxy and credentials kept outside the sandbox: the same ideas as least privilege and a restricted-network deployment.
https://www.anthropic.com/engineering/claude-code-sandboxing
Ch 3 · Incident report: Cloudflare outage on November 18, 2025 — Cloudflare, 2025-11. A timestamped public post-incident write-up (cause, detection, mitigation, restore): a model for runbooks, incident communication and the customer-facing report after a failure.
https://blog.cloudflare.com/18-november-2025-outage/
Ch 3 · Prompt caching with Claude — Anthropic (Claude blog), 2025-08. Numbers for cutting cost by up to 90% and latency by up to 85% on long prompts, the lever behind unit economics for a customer deployment.
https://claude.com/blog/prompt-caching
Ch 3 · Online migrations at scale — Stripe, 2017-02. Classic (2017): dual writes, backfill and live comparison to move data without downtime, the pattern behind zero-downtime migrations on a customer system; no newer first-party equivalent found.
https://stripe.com/blog/online-migrations

Where next

Forward Deployed AI Engineer / Applied AI Engineer · role · established

Customer-embedded engineer who scopes, builds, deploys and runs LLM systems for named customers, at AI labs and data-platform vendors. Why next: This course is that job end to end: discovery, proof of concept, evals, restricted-network deployment, handoff and field reports.

- Stronger TypeScript and Python full-stack delivery
- Agent frameworks and MCP in customer systems
- Enterprise sales cycle: technical evaluation to procurement
- Executive business cases and ROI
- Customer-site security and compliance reviews

Start it: /tutorial "Forward Deployed AI Engineer / Applied AI Engineer" basic reuse:forward-deployed-engineering-15h

Generative AI Solutions Architect · role · established

Customer-facing architect who designs and prototypes GenAI systems on a vendor platform and de-risks them with account teams. Why next: You already scope, prototype, evaluate and explain trade-offs to customers; this role adds platform depth and pre-sales architecture.

- One cloud or data platform in depth (AWS, Azure, GCP or Databricks)
- MLflow-style evaluation and monitoring
- Reference architectures and sizing
- Fine-tuning versus RAG trade-offs

Start it: /tutorial "Generative AI Solutions Architect" basic reuse:forward-deployed-engineering-15h

Agent evaluation engineering · discipline · growing

Designing eval suites, graders, judges and regression gates that tell a team whether an AI system is getting better or worse. Why next: Evals were the backbone of Part 2; this goes deeper into grader calibration, trace review and eval infrastructure.

• Calibrating LLM judges against human raters
• Trace-based and multi-turn agent evals
• Eval noise and infrastructure effects
• Online monitoring of production quality
Start it: /tutorial "Agent evaluation engineering" basic reuse:forward-deployed-engineering-15h

Harness engineering · discipline · emerging

Designing the scaffolding around a model (tools, context, verification, sandboxes, loops) so long-running agents stay reliable. Why next: Customer agents fail in the environment around the model; you have already built tools, guardrails and a deployed runtime.

• Generator and evaluator agent pairs
• Context handoff across sessions
• Removing harness parts as models improve
• Sandboxing and credential isolation
Start it: /tutorial "Harness engineering" basic reuse:forward-deployed-engineering-15h

AI gateways, sandboxing and agent security · technology · growing

Gateways and isolation layers that add auth, spend limits, caching, DLP and egress control between agents and models or tools. Why next: Your restricted-network, secrets, cost and guardrail lessons are the building blocks; these products package them.

• Gateway routing, fallback and spend limits
• Egress proxies and domain allow-lists
• Keeping credentials out of agent sandboxes
• DLP and prompt-injection containment
Start it: /tutorial "AI gateways, sandboxing and agent security" basic reuse:forward-deployed-engineering-15h

Site reliability engineering for AI systems · discipline · established

Running services against SLOs with incident command, blameless postmortems and capacity planning, applied to model-backed systems. Why next: You wrote runbooks and a provider-outage drill; SRE is the discipline that makes them routine across many deployments.

• SLOs and error budgets for AI features
• Incident command roles and comms
• Postmortem practice
• Model-provider failover and rate-limit capacity
Start it: /tutorial "Site reliability engineering for AI systems" basic reuse:forward-deployed-engineering-15h

Appendix: details for reviewers

How this syllabus was checked, and other ways to take it. Learners can skip this part.
Running example — Basket Bazaar grocery delivery issues (fictional customer): Everyone has ordered groceries or food and received the wrong item; it grows from one CSV to many stores, cities and customers. a messy orders CSV -> an API that triages one missing item -> evals and auth -> a Compose deployment -> per-store tenancy and a handover
LLM use: The course's work is building AI features for customers; a few lessons call the real model, recorded once and replayed without a key. Recorded once and replayed (no key needed by learners); at most 5 model calls per lesson.

Shorter paths through the same course

Path	Lessons	Time
Essentials	22	~10 h
Professional	33	~15 h
Complete	33	~15 h

Coverage report

41 concepts from 13 independent sources. Must-cover: 36; taught: 32; left out on purpose: 4.

Developer coverage (D1–D18)

Item	Lessons
D1 Purpose and architecture	01, 19
D2 Mental model and core concepts	01, 02, 06
D3 Setup and first working app	04, 09
D4 Day-to-day API surface	04, 05, 06
D5 Data, schemas and configuration	05, 23
D6 Integration with the app stack	04, 13, 14
D7 Async, concurrency and performance basics	N/A — this course uses synchronous calls; async Python is a prerequisite from the Python track
D8 Errors, validation and reliability	04, 28
D9 Testing and evaluation	09, 10, 11, 12, 32
D10 Debugging and observability	04, 09, 16, 28
D11 Security	13, 14, 27
D12 Packaging, deployment and operations	14, 17, 24
D13 Scaling and cost	18, 28, 29
D14 Versions, upgrades and migration	24
D15 Ecosystem and choosing	01, 25
D16 Common pitfalls and anti-patterns	03, 26
D17 Reference architectures and real use cases	21, 31
D18 End-to-end capstone	32

Enterprise depth coverage

Item	Lessons
1 distributed architecture internals	Go deeper — the internals of distributed systems belong to a systems course
2 cluster operations	Go deeper — multi-node clusters need a data-platform course; this course runs one service with CSV and Excel files
3 failure and recovery	28

Item	Lessons
4 extreme scale	Go deeper — capacity maths at 10x to 1000x needs a load-testing course
5 deployment options	14, 15, 24
6 performance engineering at scale	Go deeper — p99 load testing and capacity models need a load-testing course
7 high availability and disaster recovery	Go deeper — multi-region HA belongs to the platform course
8 zero-downtime change	24
9 multi-tenancy and isolation	23
10 security and compliance at organisation scale	13, 27
11 cost engineering	29
12 platform and organisation patterns	25
13 frontier	19
14 case studies	21

Out of scope

Fine-tuning and model training for customers — research level, and a different job; see the ML courses
Kubernetes deployment — Kubernetes is its own course; lesson 15 runs the same service with Compose

Syllabus history

Version	Date	Changes
1	2026-10-24	syllabus refresh
2	2026-10-26	wording and metadata after the packing-to-triage story change: lessons 01-03, 05, 07, 08 on the triage project; 'handoff' renamed 'handover'; lesson 28 capped per-call retry; lesson 32 time box; OpenTelemetry also used in lesson 20; lesson 24 minutes 25
3	2026-10-26	add lesson 24a: long batch jobs (restart, reconcile, finish on time), professional, 30 min, for a system-design question on a resumable nightly run